脑电数据介绍及例程

一、 数据介绍

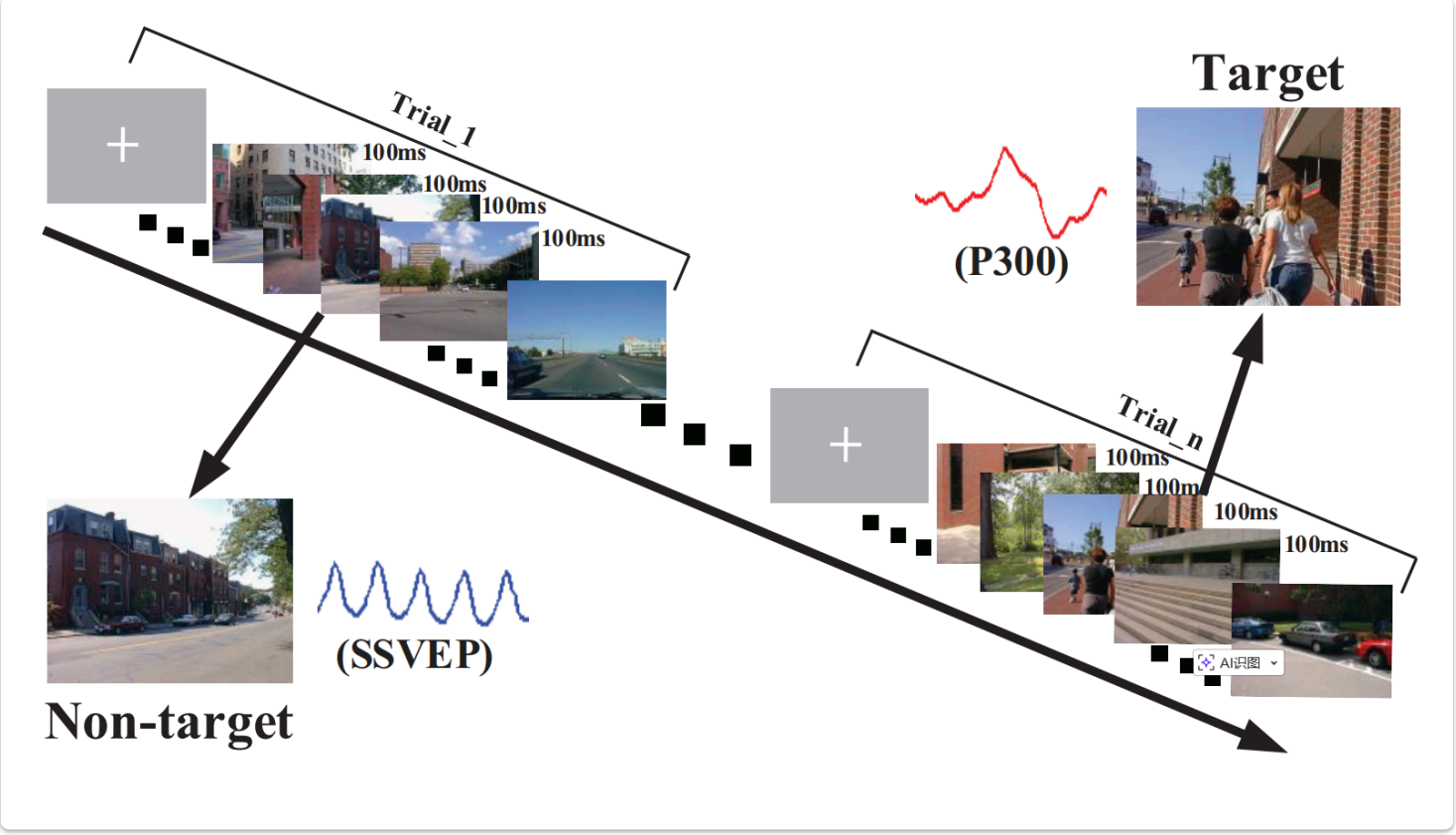
这是一份以严谨、学术的口吻撰写的数据说明，去除了口语化表述，保留了清晰的数据结构和实验范式解析，适合作为实验指导书或课件的正式背景材料：

RSVP-BCI 脑电目标检测数据集 (sub7A.mat) 数据说明

本实验所采用的数据集获取自基于快速序列视觉呈现（Rapid Serial Visual Presentation, RSVP）范式的脑机接口系统 。为了获取高质量的脑电信号，实验在严谨的物理环境中开展，并设定了极高认知负荷的视觉任务。

1. 实验背景与范式

本实验所采用的数据集获取自基于快速序列视觉呈现（Rapid Serial Visual Presentation, RSVP）范式的脑机接口系统 。受试者在实验过程中执行了视觉目标图像检测任务 。在一个标准的试次（Trial）开始时，屏幕中央首先会出现持续 0.5 秒的十字注视点，引导受试者集中视线并保持稳定 。随后，屏幕中央的灰色背景区域内，开始以 10 Hz 的高频（即每秒 10 张，每张仅呈现 100 毫秒）连续闪烁呈现真实街景图像 。受试者被要求在令人眼花缭乱的图像流中，主观搜索包含“人物”的“目标图像”，并忽略不含人物的“非目标图像” 。一旦发现目标，受试者需要按下按键以维持高度的注意力 。由于目标图像（有人物）的出现概率极低（仅为 1%~4%） ，当受试者突然捕捉到目标时，大脑会诱发出特定的事件相关电位（如 P300 等 ERP 成分） 。这种极其微弱的电位变化就隐藏在你们即将分析的数据矩阵中。



2. 数据文件结构

本实验提供的 `sub7A.mat` 包含了第 7 号受试者（A组）的未经任何处理的原始连续脑电数据。数据以双精度浮点值存储，读取该文件将解析出以下四个核心二维矩阵：

- **`EEGdata1` 与 `EEGdata2`**：分别对应实验中 Block 1 和 Block 2 的连续脑电信号 。其矩阵维度为 `[64, L]`，第一维（64）表示电极通道索引，第二维（L）表示时间点长度 。需要注意的是，不同 Block 的 L 值可能存在微小差异 。
- **`class_labels`**：类别标签矩阵，维度为 `[2, 4000]` 。矩阵中的数值“1”代表当前视觉刺激为目标图像，数值“2”代表非目标图像 。
- **`trigger_positions`**：事件触发位置矩阵，维度为 `[2, 4000]` 。该矩阵记录了每张图像呈现时，在脑电连续时间序列（长度 L）中对应的采样点索引。此参数是进行脑电数据时域分段（Epoching）的关键依据。
  -本实验中，我们关注**目标图像对应的脑电波形。**

3. 实验刺激总量与维度解析

关于标签矩阵和触发矩阵维度 `[2, 4000]` 的实验逻辑说明如下：

- 矩阵的第一维（大小为 2）分别对应实验的两个数据块（Block 1 对应第一行，Block 2 对应第二行）。
- 该组实验数据总计包含 8000 个视觉刺激周期（100 个周期/图像 × 40 个试次 × 2 个 Block）。因此，单个 Block 内部包含 4000 次图像刺激，即矩阵第二维的大小为 4000。
- 每一个周期（Circle）均对应一组 64 通道的脑电数据响应 。

数据处理建议：在后续实验中，需严格依赖 `trigger_positions` 提取对应时间窗口内的脑电片段，并结合 `class_labels` 区分目标（Target）与非目标（Non-target）试验，进而开展 ERP 提取与特征分类分析。

二、例程1

本例程将引导你完成以下简单的数据预处理流程：

1. **数据加载** (Data Loading)：读取 `.mat` 格式的原始脑电信号、通道位置信息、类别标签及打标时刻 。
2. **带通滤波** (Bandpass Filtering)：使用 Butterworth 零相位滤波器，滤除极低频基线漂移和高频环境噪声。
3. **分段提取** (Epoching)：以目标刺激出现时刻为基准点，截取前后特定时间窗内的脑电数据片段。
4. **重参考与去基线** (Re-referencing & Baseline Correction)：转换参考电极以降低共模噪声，并校正分段数据的直流偏置。
5. **叠加平均** (Averaging)：将所有目标分段数据进行叠加平均，以提高信噪比，显现 ERP 波形。

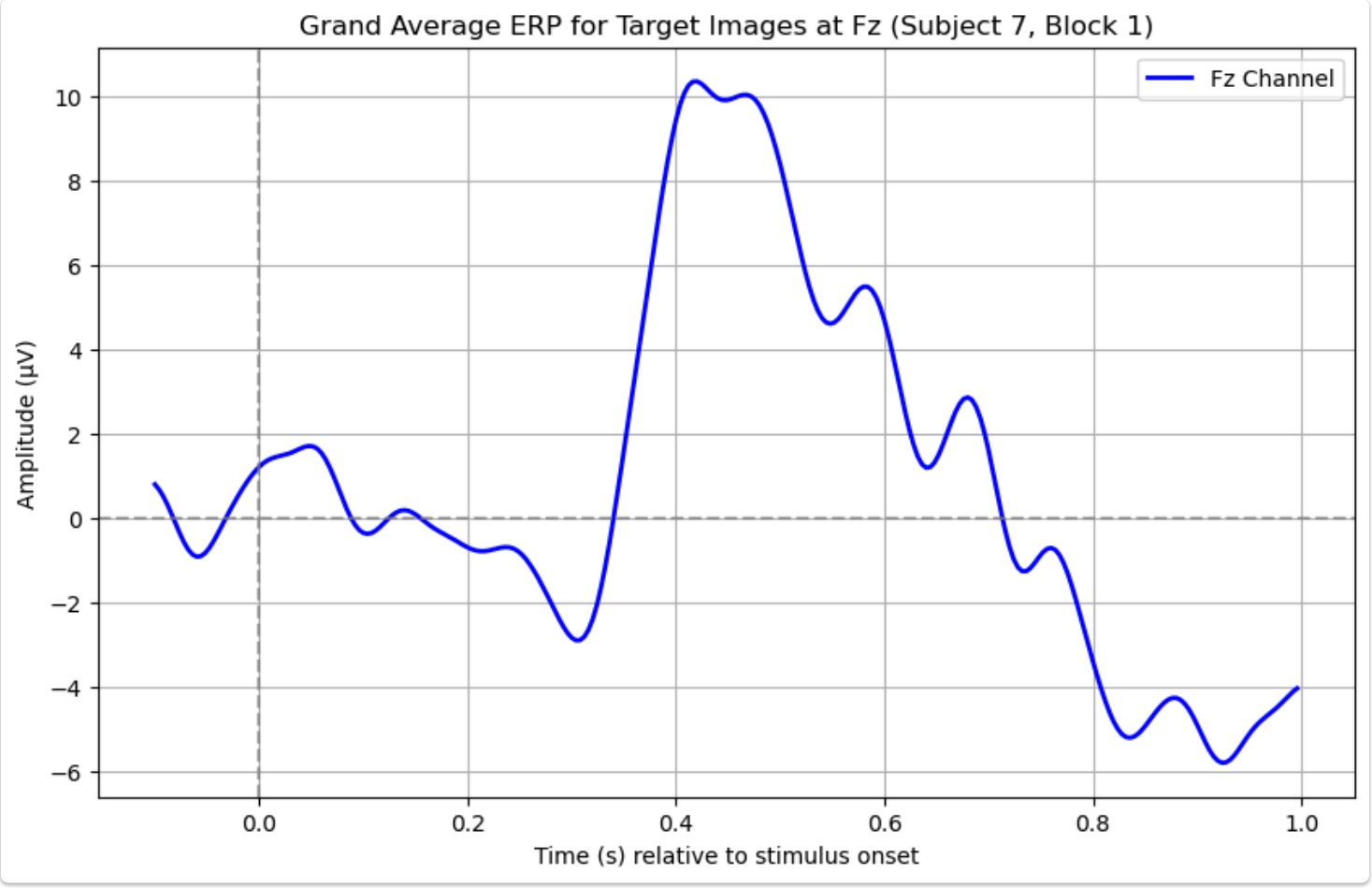
```
# %%
import numpy as np
import matplotlib.pyplot as plt
import scipy.io
from scipy import signal
```

```
import os

def read_loc_file(filepath):
    """
    手动解析 .loc 文件提取通道名称
    .loc 文件每行有四列：序号、角度、半径、通道标签
    """
    ch_names = []
    with open(filepath, 'r', encoding='utf-8') as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) >= 4:
                # 提取第 4 列（索引为 3）作为通道名称
                ch_names.append(parts[3])
    return np.array(ch_names)

def load_rsvp_data(data_path, montage_path):
    """
    加载指定受试者的 RSVP 原始数据及通道信息
    """
    mat_file = data_path
    print(f"Loading data from: {mat_file}")

    # 1. 加载 MAT 文件
    eeg_mat = scipy.io.loadmat(mat_file)
    eeg_data1 = eeg_mat['EEGdata1']          # Block 1 的连续脑电数据
    class_labels = eeg_mat['class_labels']   # 类别标签 (1: 目标, 2: 非目标)
    triggers = eeg_mat['trigger_positions']  # 刺激发生的采样点位置

    # 2. 手动读取通道名称
    ch_names = read_loc_file(montage_path)

    return eeg_data1, class_labels, triggers, ch_names

def preprocess_and_epoch(raw_data, labels, triggers, sfreq, tmin=-0.1, tmax=1.0):
    """
    对连续脑电进行滤波、分段提取目标 ERP
    """
    # 1. 带通滤波 (0.1Hz - 15Hz, 4阶 Butterworth 零相位滤波)
    order = 4
    lowcut, highcut = 0.1, 15.0
    sos = signal.butter(order, [lowcut, highcut], btype='bandpass', fs=sfreq,
output='sos')
    filtered_data = signal.sosfiltfilt(sos, raw_data)
    print("Filtering completed.")

    # 2. 分段 (Epoching) 准备
    pre_samples = int(abs(tmin) * sfreq)
    post_samples = int(tmax * sfreq)
    epoch_len = pre_samples + post_samples
```

```
    # 仅提取 Block 1 (第 0 行) 的目标触发点 (标签为 1)
    target_idx = triggers[0, labels[0, :] == 1]

    epochs = []
    # 3. 截取数据片段
    for s in target_idx:
        # 确保截取窗口不超出数据边界
        if s - pre_samples >= 0 and s + post_samples <= filtered_data.shape[1]:
            epochs.append(filtered_data[:, s - pre_samples : s + post_samples])

    epochs = np.array(epochs)
    print(f"Extracted {len(epochs)} target epochs.")
    return epochs, pre_samples

def apply_reference_and_baseline(epochs, ch_names, ref_chans, pre_samples):
    """
    应用自定义重参考及基线校正
    """
    # 1. 重参考 (Re-referencing)
    ref_idx = [i for i, name in enumerate(ch_names) if name in ref_chans]
    ref_mean = epochs[:, ref_idx, :].mean(axis=1, keepdims=True)
    epochs_ref = epochs - ref_mean

    # 2. 去基线 (Baseline Correction)
    baseline_mean = epochs_ref[:, :, 0:pre_samples].mean(axis=-1, keepdims=True)
    epochs_final = epochs_ref - baseline_mean

    print("Re-referencing and baseline correction completed.")
    return epochs_final

# ==========================================
# 主程序执行逻辑
# ==========================================
if __name__ == "__main__":
    # --- 1. 参数设置 ---
    # 请同学们修改为自己电脑上的实际路径
    DATA_DIR = r".\sub7A.mat"
    MONTAGE_FILE = r".\64-channels.loc"
    SFREQ = 250.0  # 数据采样率

    # --- 2. 加载数据 ---
    eeg_data, labels, triggers, ch_names = load_rsvp_data(DATA_DIR, MONTAGE_FILE)

    # --- 3. 滤波与分段 ---
    epochs, pre_samples = preprocess_and_epoch(eeg_data, labels, triggers,
sfreq=SFREQ, tmin=-0.1, tmax=1.0)

    # --- 4. 重参考与去基线 ---
    epochs_processed = apply_reference_and_baseline(epochs, ch_names, ref_chans=
```

```
['TP7', 'TP8'], pre_samples=pre_samples)

    # --- 5. 结果可视化：绘制叠加平均后的 Fz 导联 ERP 波形 ---
    mean_erp = epochs_processed.mean(axis=0)
    time_axis = np.linspace(-0.1, 1.0, mean_erp.shape[1], endpoint=False)

    plt.figure(figsize=(10, 6))

    # 定位 Fz 导联
    fz_idx = np.where(ch_names == 'FZ')[0][0]

    # 绘制 Fz 通道
    plt.plot(time_axis, mean_erp[fz_idx, :], 'b', label='Fz Channel', linewidth=2)

    plt.title('Grand Average ERP for Target Images at Fz (Subject 7, Block 1)')
    plt.xlabel('Time (s) relative to stimulus onset')
    plt.ylabel('Amplitude (μV)')

    plt.axvline(0, color='gray', linestyle='--', alpha=0.7)
    plt.axhline(0, color='gray', linestyle='--', alpha=0.7)

    plt.legend()
    plt.grid(True)
    plt.show()
```

运行结果如下图所示：



三、例程2

由于真实脑电信号成分复杂，**实验四 - (2) 的实验现象，在真实脑电信号上表现不明显，下面附上模拟数据的实验程序。**

```
import numpy as np
from scipy import signal
import matplotlib.pyplot as plt

fs = 1000.0
t = np.linspace(-0.2, 0.8, int(1.0 * fs), endpoint=False)
# ==========================================
# 带有尖锐的 P1/N1 和宽大的 P3
# ==========================================
p1 = 0.8 * np.exp(-((t - 0.1)**2) / (2 * 0.015**2))  # 尖锐的早成分 P1 (约 100ms)
n1 = -1.2 * np.exp(-((t - 0.15)**2) / (2 * 0.02**2)) # 尖锐的早成分 N1 (约 150ms)
p3 = 2.0 * np.exp(-((t - 0.35)**2) / (2 * 0.06**2))  # 宽大的晚成分 P3 (约 350ms)
erp_book = p1 + n1 + p3 # 叠加成为基准波形
# ==========================================
# 2. 模拟图 B：滚降温和的高通滤波器 (12 dB/octave)
# ==========================================
b_mild, a_mild = signal.butter(2, 2.5, btype='high', fs=fs)
erp_mild = signal.lfilter(b_mild, a_mild, erp_book)
# ==========================================
# 3. 模拟图 C：滚降极陡的高通滤波器 (48 dB/octave)
# ==========================================
b_steep, a_steep = signal.butter(8, 2.5, btype='high', fs=fs)
erp_steep = signal.lfilter(b_steep, a_steep, erp_book)
# ==========================================
# 4. 绘图
# ==========================================
fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=True)
# [左图] 图 B：温和滚降
axes[0].plot(t, erp_book, 'k', linewidth=2.5, label='Unfiltered ')
axes[0].plot(t, erp_mild, 'b--', linewidth=2, label='Filtered (12 dB/octave)')
axes[0].set_title('Figure B: Mild Roll-off (12 dB/octave)')
axes[0].text(0.35, -0.6, 'Artifactual\nUndershoot', color='blue', ha='center')
# [右图] 图 C：陡峭滚降
axes[1].plot(t, erp_book, 'k', linewidth=2.5, label='Unfiltered ')
axes[1].plot(t, erp_steep, 'r--', linewidth=2, label='Filtered (48 dB/octave)')
axes[1].set_title('Figure C: Steep Roll-off (48 dB/octave)')
axes[1].text(-0.05, 0.5, 'Ringing!', color='red', ha='center')
for ax in axes:
    ax.set_xlabel('Time (s)')
    ax.set_ylabel('Amplitude')
    ax.set_xlim(-0.1, 0.7)
    ax.axhline(0, color='gray', linestyle=':')
    ax.axvline(0, color='gray', linestyle=':')
    ax.legend()
    ax.grid(True)
```

```
plt.tight_layout()
plt.show()
```